Smart Mobility Hub — Engineering Report (DRAFT)

Smart Mobility Engineering Lab, Prof. Mehdi Pirahandeh — Inha University ISE

Team Shenanigans — team leader: Egamberdiev Saidumar

Members: Husan (M1-A), Tuymurod (M1-B), Fazliddin (M2-A), Arkar (M2-B), Carla (M3), Aidil (M4), Saidumar (M5)

Platform: ROS 2 Jazzy, Ubuntu 24.04, Python rclpy, Docker (verified on Apple Silicon aarch64)

Source: branch integration/current-team-work (included in the submission ZIP); evidence files record their commit

This is a working draft. Sections for M1-A, M1-B and M5 contain measured results from the repository. Sections marked **TODO** belong to members whose code is not yet in the repository; they must be completed with real results before submission. No result in this report is invented — every number comes from a file in `evidence/` or `src/smart_hub/results/`. Everything is **simulation only**; no physical hardware was used.

1. Requirements and scope

The assignment asks for a simulation-first transport hub in ROS 2: passenger detection (S1), AI demand prediction (S2), bus docking (S3), parking (S4), congestion detection (S5), route replanning (S6), robot escort (S7), integrated coordination (S8) and a negative/fault suite with emergency stop (S9), each verified by tests T01–T18 (§21). The safety rule that shapes the whole design: managers only *publish candidate velocities*; only `safety_supervisor` publishes final motor commands (§4.2).

Scenario	Owner	State at the evidence commit
S1 passenger detection	M1-A	implemented, verified in simulation; real-video evaluation pending
S2 demand prediction	M1-B	implemented, T03 evaluated on held-out seeds
S3 bus docking	M2-A	S3 12/12 live, T05 10/10, T06 passed (code by M5, review by M2-A pending)
S4/S5 parking, congestion	M3	nodes in the ROS package with unit tests; S4 11/11 and S5 9/9 pass live
S6/S7 routing, escort	M4	S6 8/8, S7 9/9, T18 5/5 live (code by M5, review by M4 pending)
S8 integration	M2-B (+ all)	T15 passed: 20 rush-hour episodes (10 seeds × predictive/reactive), 0 safety violations, 100 % completion
S9 faults, e-stop	M5	implemented, verified in simulation (T16, T17, 25-trial repetition)

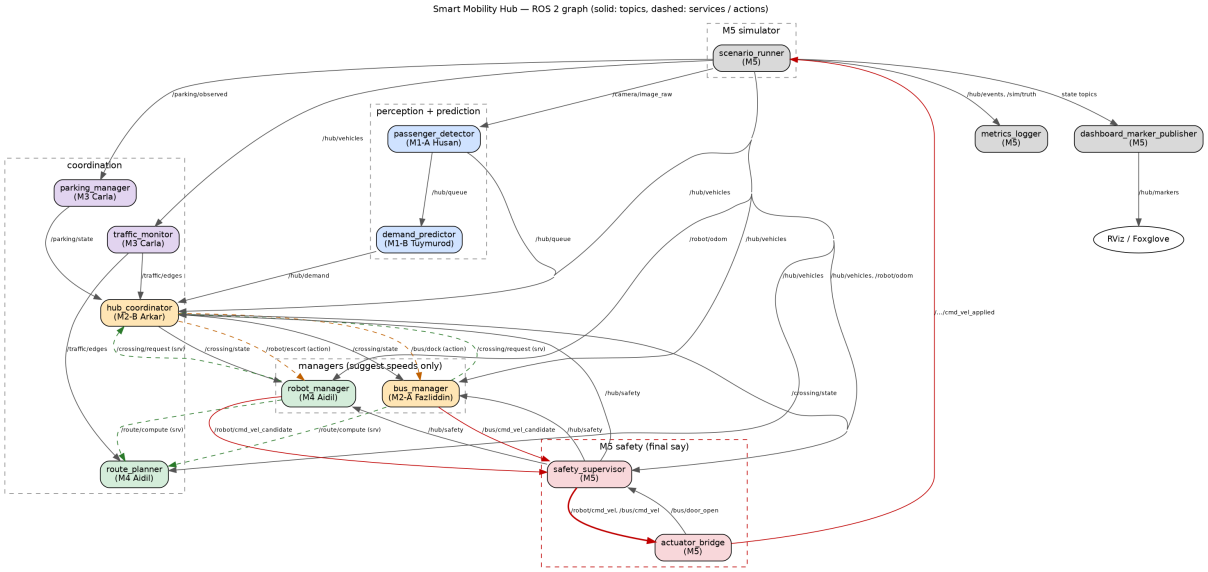


Figure 0 — ROS 2 graph: nodes by owner, topics (solid), services and actions (dashed); the managers' candidate velocities pass only through safety_supervisor and actuator_bridge (red).

2. Map, coordinates and frames

Map M1 is a 12 m × 8 m site, origin at the south-west corner, +x east, +y north, metres and radians (§2.1). Queue Q is x ∈ [2, 6], y ∈ [5, 7]; crossing C is x ∈ [4.5, 5.5], y ∈ [1.25, 4.75]; obstacle O is centred at (7, 2) with radius 0.35 m. Graph edge ids are written as sorted node names (`Cs-J`, `E-J`), a team convention shared by the simulator and the traffic monitor. Frames: `map`, `robot/odom`, `robot/base_link`; map→odom is identity in the simulator. Vehicle ids are `robot_1` and `bus_1`.

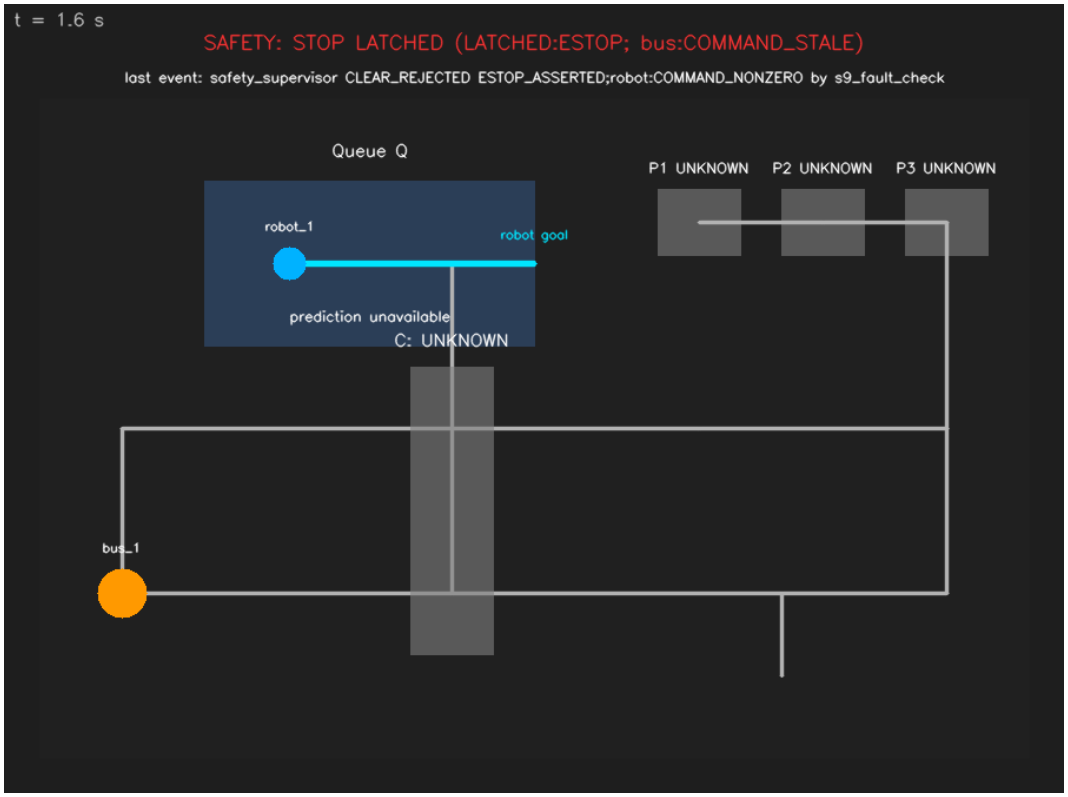


Figure 1 — Dashboard (/hub/markers rendered without RViz) during S9: robot with its route in queue Q, bus at entry A, crossing and parking UNKNOWN because those nodes were not running, safety latched by the e-stop.

3. ROS graph and interface rationale

All custom types are in `smart_hub_interfaces` (14 messages, 6 services, 3 actions, copied field for field from §6–7). The full topic/service inventory with rates, QoS and owners is in `docs/INTEGRATION_CONTRACT.md`.

Interface	Kind	Why this kind
`/hub/queue`, `/hub/demand`, `/hub/vehicles`, `/robot/odom`	topics	continuous state streams with several consumers
`/robot/cmd_vel_candidate` → `/robot/cmd_vel`	topics (R1)	only the newest command matters; the supervisor is the single gate
`/hub/estop`, `/hub/safety`	topics (reliable, transient-local)	a node that starts late must still learn that a stop is active
`/safety/clear_stop`, `/scenario/reset`	services	short request/response with an explicit accept/reject reason
`/bus/dock`, `/robot/escort`	actions	long missions with feedback, cancellation and a result
`/robot/route`, `/bus/route`	topics (`nav_msgs/Path`)	team decision: planned routes for the dashboard and RViz

Every state message is stamped with its *observation* time, and consumers judge freshness by both stamp and receipt time; unstamped or future-stamped data is never fresh (§5).

4. M1-A — passenger detection and queue counting (S1)

Pipeline. `/camera/image_raw` → person boxes → foot point (bottom centre) → planar homography from 4 calibrated floor points (DLT) → nearest-neighbour tracking in metres → count of confirmed tracks inside Q, crossing flag for C → `/hub/queue` at 5 Hz. Two detectors: a documented dark-region baseline (`blob`) for the synthetic camera and OpenCV's pretrained HOG people detector (`hog`) for real video (§8.2 step 1).

Rules (§8.2). 0.4 s appearance and disappearance debounce; identity kept through 0.5 s occlusion; crossing held occupied until C is clear for 2 s; feed silence or a frozen source stamp for 1 s makes the state invalid (count 0, confidence 0, `valid=false`, crossing reported occupied so nobody is granted entry).

Results (simulation, synthetic camera from M5).

Test	Criterion (§8.3)	blob baseline	HOG
T01 stable counts	0, 3, 5, 4 after debounce	0, 3, 5, 4; MAE 0.000 over 127 samples	MAE 0.992 (≤ 1 target met narrowly)
T02 no double count	0.3 s occlusion creates no new id	6 tracks for 6 people	4 tracks for 6 people (recall 0.67)
T02 freeze	invalid within 1.2 s	1.034 s	1.03 s
crossing hold	occupied while in C + 2 s	on after 0.58 s, clear 2.48 s after leaving	same

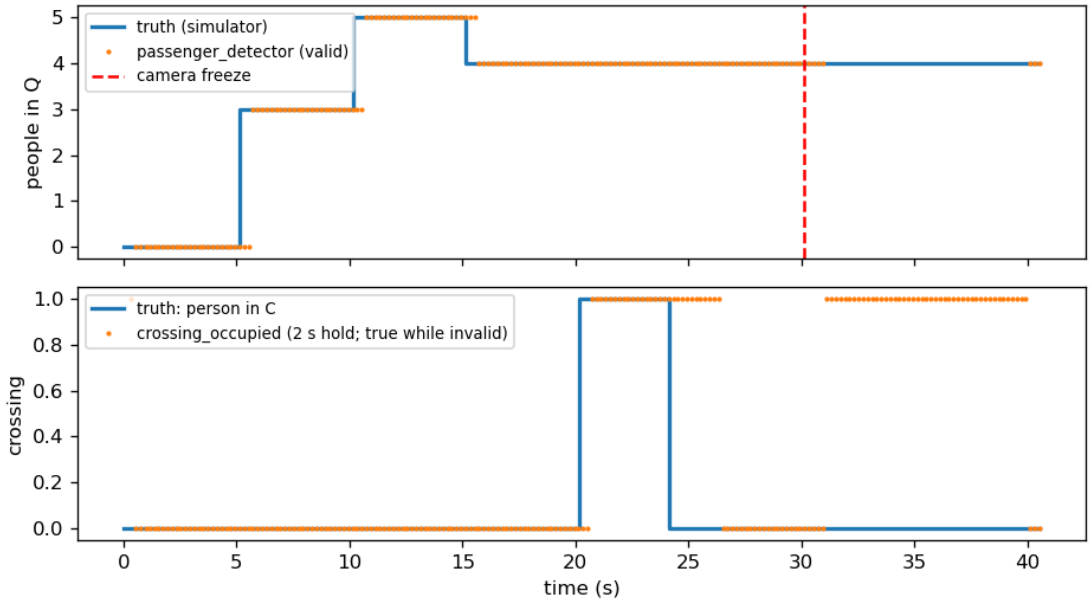


Figure 2 — S1 count-versus-truth (blob baseline). Top: queue count; bottom: crossing flag with the 2 s hold and the conservative "occupied" while invalid after the camera freeze at 30 s.

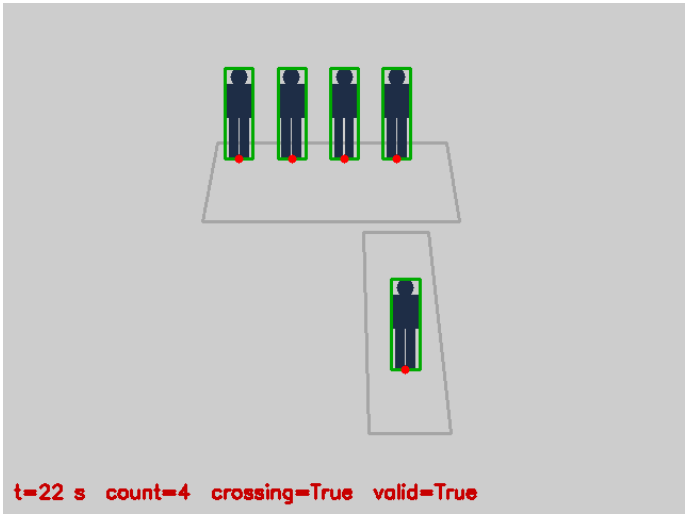


Figure 3 — Annotated synthetic camera frame at t = 22 s: detection boxes, foot points (red), four people in Q and one in C.

A failure found and corrected. The count-versus-truth plot showed that, right after a camera freeze, the count decayed to a *valid* zero for about 0.6 s before the state became invalid, because the disappearance debounce kept running without frames. The count is now judged at the last processed frame; a regression test (`test_silent_feed_never_reports_a_valid_zero`) prevents it from returning. After the fix: 0 valid-zero samples after the freeze.

TODO (M1-A): the §8.3 real-video evaluation — record the lab video, hand-label at least 100 frames, run `video_eval`, and report count MAE, precision/recall and occlusion behaviour here, with annotated before/during/after frames. Tooling is ready and tested (MAE 0.0 on a video rendered from S1).

5. M1-B — AI demand prediction (S2)

Source: `src/smart_hub/results/S2_report.md` and `metrics.json` (synthetic seeded episodes). 50 six-minute episodes over 7 demand profiles, split by whole episode 30/10/10 before windowing; features `q_t`, 10 s and 30 s means, 10 s trend, 30 s arrivals and departures; standardized ridge regression (alpha 30 chosen on validation); persistence baseline. Target: unserved demand at t + 30 s.

T03 (2,500 held-out forecasts)	MAE	RMSE
Ridge, demand target (shipped)	1.215	1.636
Persistence baseline	1.506	2.091

Ridge beats persistence by −0.292 MAE (95 % CI −0.543 … −0.044), better in 7 of 10 test episodes. Inference p95 0.013 ms (target < 100 ms). In a paired closed-loop study over 50 seeds, predictive dispatch lowered mean passenger wait from 73.7 s to 63.7 s (−10.0 s, 95 % CI −14.9 … −6.0). Live in the integrated stack, `/hub/demand` ran at 1.0 Hz on M1-A's `/hub/queue` and became valid after the 60 s warm-up.

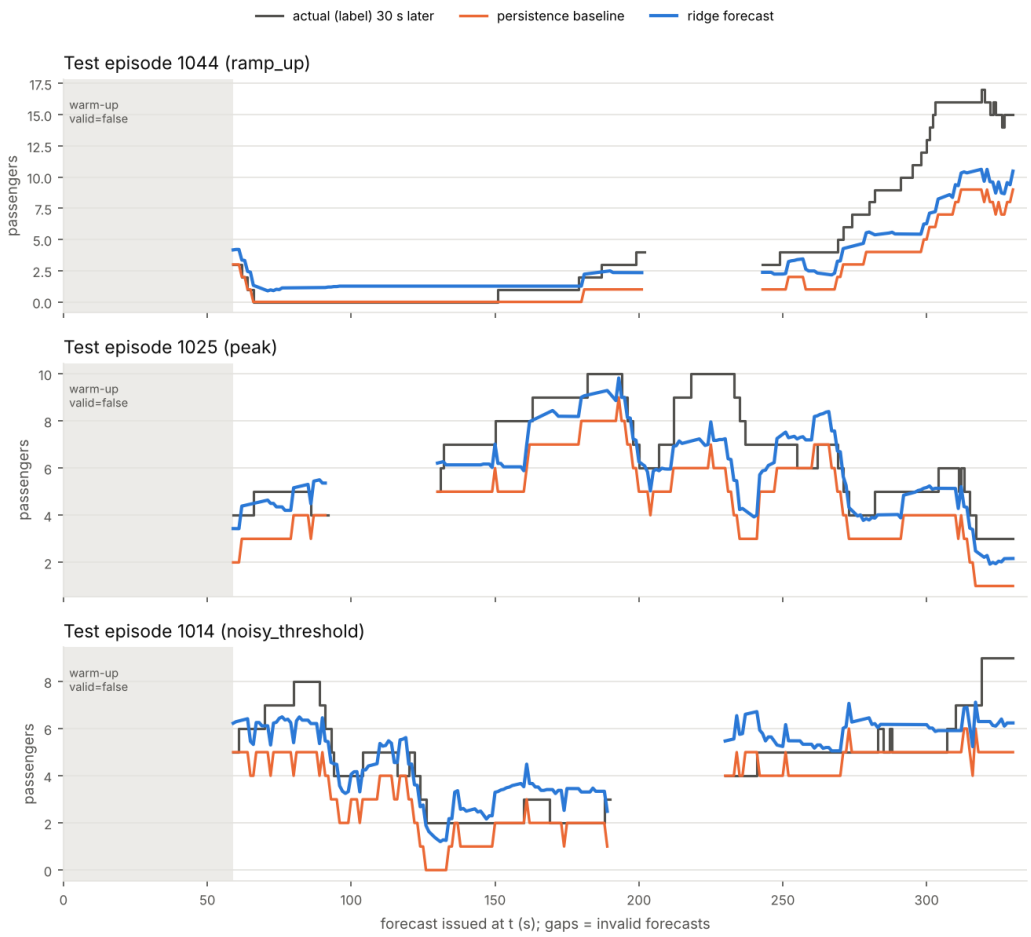


Figure 4 — S2 forecast examples (M1-B).

T04 was run with the coordinator (section 7) using a checker in place of the forecaster. The predictor now takes part in the scenario reset handshake: a reset drops its 60 s history so the next forecast is invalid until warm-up completes, and its files pass `ament_flake8` / `ament_pep257`.

6. M2-A — bus manager and docking (S3)

Draft written by M5, who also wrote the M2-A code for Fazliddin; Fazliddin (M2-A) reviews the code and confirms or corrects this section before submission.

`bus_manager` serves the DockBus action and runs the §10.2 state machine IDLE → APPROACH → (WAIT) → DOCK → BOARD → COMPLETE, with SAFE_STOP from every moving state; the logic lives in `bus_core.py` (pure Python, 10 closed-loop tests against the simulator). The bus asks M4's `/route/compute` for its route (1 s timeout, at most 2 retries, then PLANNER_UNAVAILABLE; NO_ROUTE fails at once; a fixed A-N-Cn-B fallback exists only as an off-by-default parameter), asks `/crossing/request` for C 2.5 m before a stop line 0.25 m in front of C, renews every 1 s (3 s lease) and releases once its footprint has left C; when bay B is blocked it waits before C without holding the crossing. Docked = position ≤ 0.20 m, yaw ≤ 0.17 rad and speed < 0.02 m/s for 1 s; the door is requested only then, kept open 8 s from the measured opening, and propulsion needs a measured closed door. ETA = remaining route length / smoothed measured speed + remaining turn time + a declared 1.5 s final-approach allowance; invalid while waiting.

S3 live (`docker/hub.sh s3`, `evidence/m2a/s3/`, 12/12 checks, Figure 5). T05: 10/10 dockings, route A-N-Cn-B from the planner in all 15 trips, position error 0.013–0.034 m, yaw error ≤ 0.006 rad, door open 8.1 s per stop with no movement while open, crossing granted before entry and

released after exit in all 10, no collision, 4 passengers boarded and 1 left waiting each time. ETA error at 5 m remaining: mean 0.24 s, maximum 0.94 s (target ≤ 3 s); ETA invalid 0.1 s per trip (before a credible speed). T06: zero `/bus/cmd_vel` 0.001 s after the cancel was accepted (limit 0.5 s), status CANCELED, crossing released. A pedestrian in C made the coordinator deny with `PEDESTRIAN_PRESENT`; the bus waited outside C and docked after the person left. A car in bay B gave WAIT with `BAY_BLOCKED` outside C. Freezing the vehicle feed gave a zero candidate after 0.535 s (pose timeout 0.5 s) and SAFE_STOP, then the bus resumed and docked. With seven people in Q the coordinator dispatched the bus by itself (observed-queue fallback) and the mission succeeded. The simulated bus has no inertia, so stopping times are shorter than on hardware.

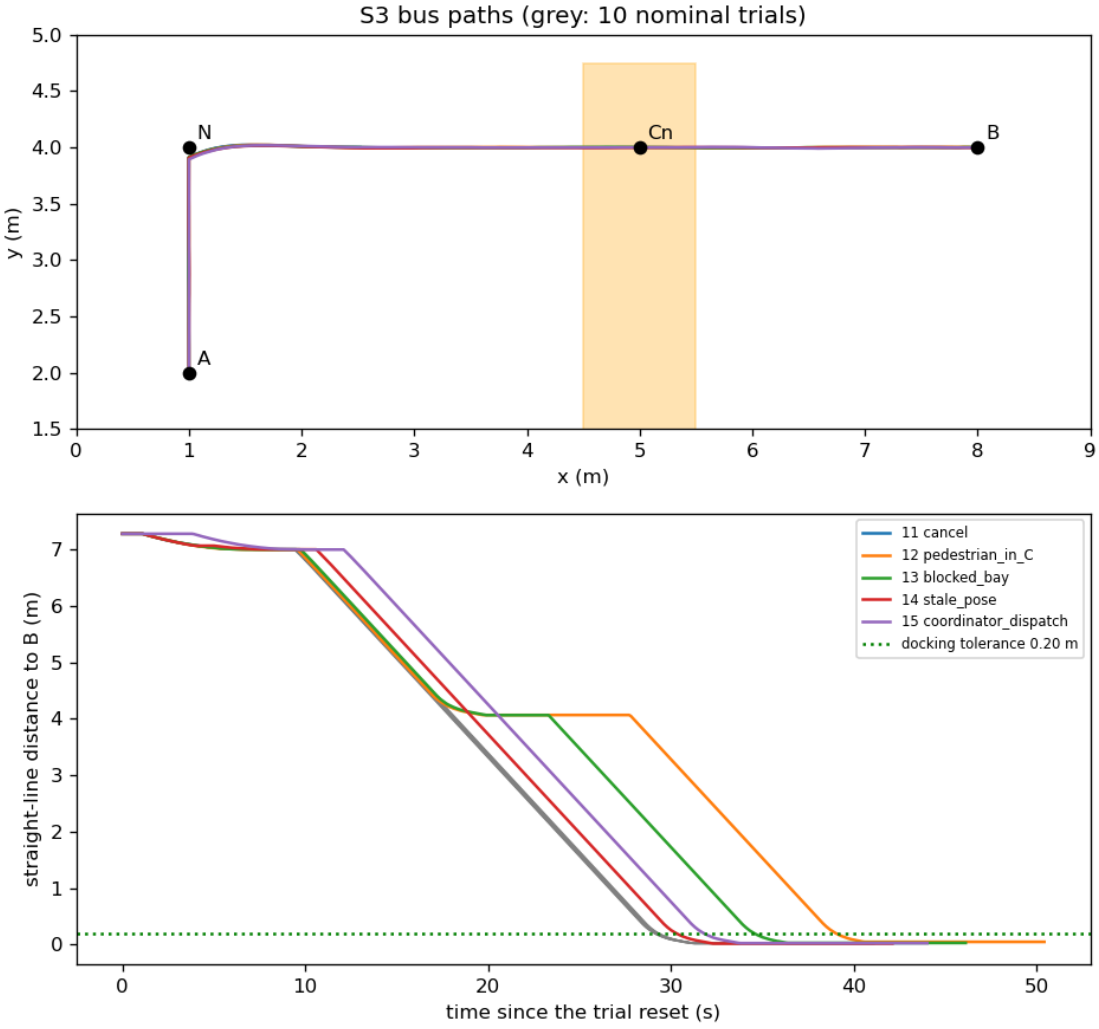


Figure 5 — S3: bus paths (top) and straight-line distance to bay B over time (bottom). Grey: 10 nominal trials; the flat segments at about 4 m are the bus waiting before crossing C (pedestrian in C, blocked bay).

Unit tests. `test_bus_core.py` (10 tests) drives the pure state machine in closed loop with the M5 simulator and no ROS: nominal docking within tolerance, crossing acquired before entry / renewed / released after exit, denied crossing → WAIT at the stop line, blocked bay → WAIT without asking for the crossing, cancel → zero command at once, stale pose → hold and resume (and abort after 5 s), ETA error at 5 m, timeout and safety latch, path geometry and parameter validation. `test_sim_core.py` covers boarding (at most four waiting people, only while the door is open and the bus is still at B), and `test_reset_launch.py` checks that bus_manager acknowledges resets and rejects a wrong bus id, an unknown bay and a zero duration.

Problems found and fixed during development. (1) The arbiter answers `BAD_TOKEN` for a permission that is already gone; the first version expected a different reason and would have retried RELEASE forever. (2) A permission lost while the bus was inside C would have made it

brake in the middle of the crossing; it now never stops inside C on its own. (3) After a stale-pose stop the bus resumed as APPROACH even when it had been boarding; it now resumes the phase it was in. All three were found by review and the closed-loop tests before the live run.

Limitations. Departure after COMPLETE (a separate transition, §10.2 step 6) is not implemented; in the tests the scenario is reset and the bus starts at A again. Only bay B is supported. Without an answer from `/route/compute` the bus does not move (T18). A cancel while inside C stops the bus there and keeps renewing the crossing until a new goal drives it out (not tested live). The simulated bus has no inertia.

7. M2-B — hub coordinator and S8 integration

`coordinator_core.py` implements crossing arbitration (one owner, 2 s clear debounce, lease HOLD instead of FREE) and the §9.3 dispatch hysteresis; 28 unit tests pass. Verified against the shared interfaces in a scratch workspace: a crossing was granted while `/hub/safety` was fresh and unlatched, and denied with `SAFETY_STOP` after an e-stop. The node runs inside `hub.launch.py` with the rest of the hub; in S3 it granted and released crossing C for the bus and, with seven people in Q, dispatched the bus by itself.

T04 live (`docker/hub.sh t04`, `evidence/m2b/t04/`, 6/6 passed). A checker publishes `/hub/demand` at 1 Hz and `/hub/queue` at 5 Hz; the real coordinator and bus_manager do the rest. Forecasts alternating around 6 (6.2, 5.8, 6.1, …) for 20 s gave no dispatch. A rising forecast gave exactly one dispatch (reason PREDICTED) 2.0 s after the first value ≥ 6, i.e. on the third consecutive sample; bus_manager accepted one goal and saw no second request while it was outstanding, and none followed in the 10 s after the mission while the forecast was 5 (escalated but below 6). `valid=false` forecasts (predicted 9) and a feed gap with an observed queue of 2 gave no dispatch; with no forecast and an observed queue of 7 the coordinator dispatched once with reason FALLBACK. During the work `demand_freshness_sec` (2.5 s) was added: the forecast arrives at 1 Hz, so with the shared 1.0 s freshness limit a forecast would go stale exactly when the next one was due (found by review, not observed).

S8 / T15 — integrated rush hour (`docker/hub.sh s8`, `evidence/s8/`, **T15 passed**, Figures 9 and 10). Every node runs together: camera → passenger_detector → demand_predictor → hub_coordinator → bus_manager, plus parking_manager, traffic_monitor, route_planner, robot_manager and the M5 stack. The script (`scenarios/S8.yaml`) follows §15.1: warm-up with three passengers, arrivals rising so the observed queue reaches 7 at about 61 s, two cars requesting parking at the same moment at 65 s, an escort request at 70 s (a kiosk event, see below), the passenger's ready press at 75 s, a pedestrian in C from 80 to 85 s, obstacle O on Cs-J from 90 to 120 s, and the remaining passengers walking away from 160 s; each seed jitters the arrival times. After docking the bus departs to exit E as a separate, separately reported transition. Ten seeds were run in both dispatch modes, each episode in a fresh launch (20 episodes × 240 s).

	predictive	**reactive**
safe episodes (0 collisions, 0 crossing conflicts, 0 double allocations, 0 duplicate dispatches)	10/10	10/10
completion (bus docked + departed, escort arrived, both parking requests answered)	10/10	10/10
dispatch time, mean (range)	62.7 s (61.8–65.6)	64.1 s (62.8–64.8)
dispatch reason	PREDICTED	FALLBACK (observed queue)
queue wait to boarding, p50 / p95 (40 boarded passengers)	77.5 s / 98.4 s	79.2 s / 99.2 s
bus travel A→B / robot escort ready→arrival, mean	37.8 s / 82.3 s	37.4 s / 83.3 s
crossing waiting time per episode, mean	9.8 s	9.5 s

In all 20 episodes the forecast became valid only after warm-up (59.7–62.7 s); exactly one bus mission was dispatched; C1 and C2 asking for P2 at once received P2 and P3; the bus was granted C first and the robot after it (never overlapping, never an owner change without release); both

paused for the pedestrian in C; and the robot rerouted around O (`W-Cn-B-U-E-J-D` or `Cn-B-U-E-J-D` depending on where it was) and arrived. Paired by seed, predictive dispatch was earlier in 7 of 10 seeds, later in 2 and equal in 1; the gain is small (1.4 s earlier dispatch, 1.7 s lower median wait) because in this script the queue jumps from 3 to 7 within about 9 s, so the 30 s forecast has little lead, and the waits are dominated by passengers who arrived during warm-up plus the 38 s bus trip. Capacity is 4, so 3 passengers per episode were not served before walking away; they are excluded from the wait statistics (censored), not counted as zero. These are simulation results on one fixed rush-hour shape; they do not establish a transport benefit beyond it.

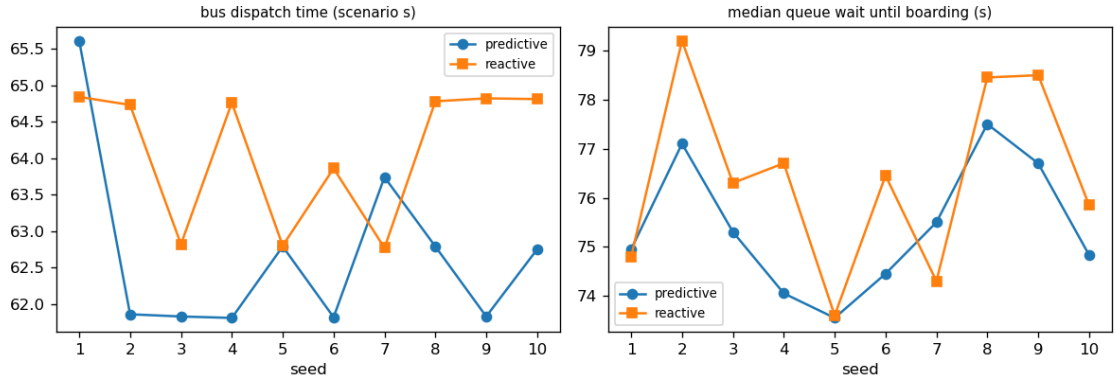


Figure 9 — S8 / T15: bus dispatch time and median queue wait per seed, predictive vs reactive dispatch on identical seeds.

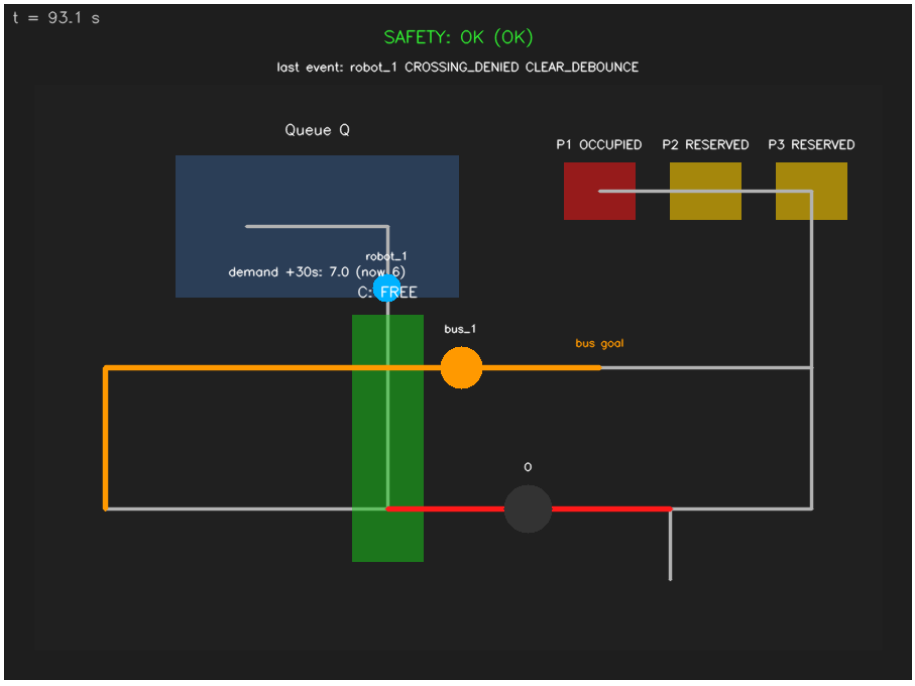


Figure 10 — S8 dashboard at t = 93 s (seed 1, re-recorded after the dashboard fix below; the measured episodes are unaffected): the bus has just left crossing C, the robot waits at its stop line for the 2 s clear debounce, obstacle O blocks Cs-J (red), P2 and P3 are reserved by C1 and C2, forecast 7.0 for +30 s.

Escort trigger (open question answered). The assignment defines no interface for "escort request at 70 s". A kiosk publishes `ESCORT_REQUEST` on `/hub/events` with JSON `{request_id, passenger_id, pickup, destination}` (in simulation the scenario_runner does it); the coordinator turns it into one EscortPassenger goal, and the simulated passenger button calls `/robot/confirm_ready`. The coordinator gained a `dispatch_mode` parameter (predictive / reactive) for the comparison.

One difficult integration failure and its correction. The first S8 episode showed zero safety violations, yet the bus and the robot never moved: the bus timed out and the escort never

finished. The log showed the actuator bridge exiting at start-up with "only mode:=sim is implemented". The S8 launch file had a `mode` argument (predictive / reactive), and ROS 2 launch arguments share one scope with included files, so `mode:=predictive` also reached hub.launch.py's `mode` (sim / hardware) and the bridge refused to drive — the fail-safe behaved as designed, but the whole run was meaningless. The argument was renamed `dispatch`; the next episode completed every mission. The same integration pass found three more problems: (1) a parked bus counted as traffic on its own edge (route_planner, section 9); (2) the dashboard dropped the 1 Hz forecast after 1.0 s and flickered to "prediction unavailable" (now a 2.5 s limit for the forecast, like the coordinator; display only); (3) the T15 batch script hung after the first episode because a background rosbag recorder without job control ignores SIGINT (`set -m` added). Robot and bus also needed one behaviour added for §15.1: an agent that already owns C pauses while a pedestrian is reported in it (holding before C, or standing still inside), because the arbiter only blocks *new* grants.

8. M3 — parking and traffic (S4, S5)

> Draft written by M5 from M3's code and the measured runs; Carla (M3) confirms or corrects it before submission.

Both subsystems follow one pattern: the logic lives in a ROS-free engine (`parking_engine.py`, `traffic_engine.py`, map geometry in `hub_map.py`) that receives the current monotonic time as an argument, and a thin node (`parking_manager`, `traffic_monitor`) adapts it to topics, services, timers and QoS. Both nodes run inside `hub.launch.py`.

Parking (S4). The engine keeps the *physical* state (debounced sensor reading) and the *administrative* state (reservation token, vehicle, expiry) apart and derives the effective state on demand: UNKNOWN if the slot was never observed, is older than 1.0 s or a contradicting reading is still being debounced (0.5 s); OCCUPIED if sensed occupied (ownership is kept but never shown as free); RESERVED if a live token exists; FREE otherwise. Only a fresh, debounced FREE slot is allocatable, so UNKNOWN or OCCUPIED slots are never advertised as free (§6.1). Every mutation — validation, request-id cache, slot choice, token creation, expiry — runs under one re-entrant lock, and the two services share one mutually exclusive callback group, so two requests can never interleave (atomicity, §18.2). `request_id` is an idempotency key: the same id with the same payload returns the cached answer, a different payload returns `KEY_REUSE`. The slot is the preferred one if free, otherwise the free slot nearest to access node U by graph distance (P3, P2, P1). One active reservation per vehicle; TTL 5–120 s (default 30 s). Release needs the token and the matching vehicle (`UNKNOWN_TOKEN`, `VEHICLE_MISMATCH`). Expiry and release drop only the administrative reservation and never change the sensed state. A barrier controller opens `/parking/barrier_request` only for a live reservation presented by its own vehicle with a fresh, clear entrance sensor, and closes only after a hold time with the passage continuously clear. Snapshots older than 1.0 s or stamped in the future are dropped, so slots age out to UNKNOWN by themselves.

Traffic (S5). Every `/hub/vehicles` snapshot (10 Hz) is gated: the stamp must advance (a repeated stamp is a frozen source), the source age must be within 1.0 s, and each vehicle needs a finite pose, a plausible speed (≤ 1.0 m/s), a map edge id and a position inside that edge's 0.5 m corridor (no silent re-assignment). Obstacles (`kind: obstacle`) are intersected with the corridors. Every 0.5 s each edge gets one label: UNKNOWN when there is no fresh data for more than 1.0 s, during warm-up (3 fresh snapshots) or when a sensor marks a vehicle on that edge invalid; BLOCKED while an obstacle intersects the corridor (kept 2 s after it disappears); CONGESTED when density ≥ 0.70 (n / floor(length / 1.0 m headway)) or the mean of per-vehicle median speeds stays below 40 % of free flow (0.40 m/s) for 3 publications, cleared after 2 publications below; NORMAL otherwise, including an empty edge with a live feed. Invalid edges carry speed and density −1.0, never 0, and the header stamp is the source time of the newest fresh snapshot so consumers can age-check it. A 10 Hz watchdog publishes UNKNOWN as soon as the feed goes

stale instead of waiting for the next 2 Hz slot. `/traffic/edges` (EdgeStateArray) has no label field, so labels are also published as JSON on `/traffic/edge_labels` and as `/hub/events` on every transition.

Unit tests (T07–T10). `test_t07_parking_atomicity.py` (5 tests, including a forced race through a hook inside the reservation critical section), `test_t08_parking_expiry.py` (28: expiry of occupied and stale slots, release validation, idempotency, TTL limits, barrier rules), `test_t09_congestion.py` (9: density and speed rules, hysteresis, obstacles) and `test_t10_stale_state.py` (17: frozen stamps, gaps, invalid vehicles, recovery) run inside `colcon test`, plus a node smoke test; all pass (the stub-based smoke tests are skipped when real ROS is present).

S4 live (`docker/hub.sh s4`, `evidence/m3/s4/s4_check.json`, 11/11 passed). A checker node plays the cars and calls the real services. C1 and C2 asked for P2 at the same moment and received P2 and P3 with different tokens; repeating C1's request id returned the same reservation; a wrong token was rejected with `UNKNOWN_TOKEN` and C2 using C1's token with `VEHICLE_MISMATCH`; C1's slot became OCCUPIED 0.64 s after it was sensed (0.5 s debounce); P3 became UNKNOWN 0.18 s after its sensor went offline; after both reservations expired P2 stayed OCCUPIED and P3 UNKNOWN (expiry never freed an occupied or stale slot); with no free slot a request failed with `FULL`. T07: 20 synchronized request pairs gave 0 double allocations, p95 service latency 39 ms (limit 250 ms), maximum 133 ms.

S5 live (`docker/hub.sh s5`, `evidence/m3/s5/`, 9/9 passed, Figure 6). On edge Cs-J (capacity 4): with one car at 0.35 m/s every label was NORMAL (34 labels, no false congestion); three slow cars joining gave CONGESTED after 0.29 s, held while they stayed; NORMAL 1.8 s after they left. T10: freezing the vehicle feed made every edge UNKNOWN after 0.995 s (limit 1.2 s) with speed and density −1.0, and valid again 0.3 s after the feed resumed. Obstacle O gave BLOCKED after 0.33 s (within one 2 Hz publication) and NORMAL 2.3 s after removal (2 s obstacle-clear hold). The UNKNOWN margin is only 0.2 s because the monitor's stale threshold is 1.0 s.

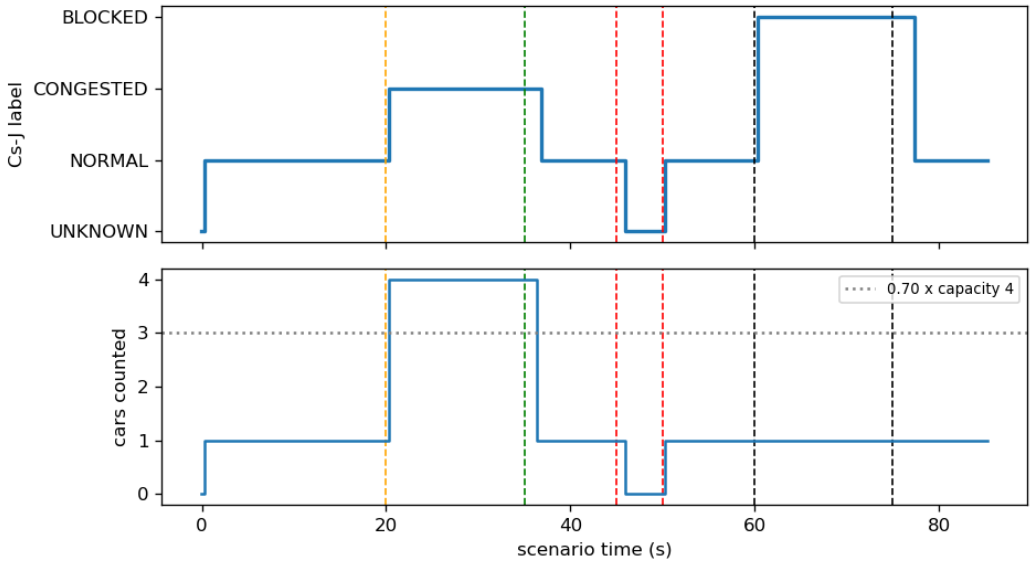


Figure 6 — S5 label of edge Cs-J (top) and cars counted on it (bottom). Dashed lines: slow cars join (orange) and leave (green), vehicle feed frozen and resumed (red), obstacle on and off (black).

The scenario files and both checkers were written by M5 (simulator owner). Both node `main()` functions were changed during integration so that Ctrl+C no longer raises `RCLError: rcl_shutdown already called` (the code paths above are unchanged).

Limitations. A vehicle standing still counts as slow, so a bus parked at A for a while is labelled `A-Cs: CONGESTED` (a stop is not distinguished from a jam). The UNKNOWN latency margin is 0.2 s (stale threshold 1.0 s, limit 1.2 s). The barrier interlock topics (`/parking/entry_claim`, `/parking/entrance_clear`) are additions not in the assignment and are not exercised by S4.

Parking performance was measured in simulation on one machine; p95 includes the ROS service round trip from a client in the same container.

9. M4 — routing and robot escort (S6, S7)

> Draft written by M5, who also wrote the M4 code for Aidil with his permission; Aidil (M4) reviews the code and confirms or corrects this section before submission.

Planner (S6). `route_planner` serves `/route/compute`; the logic is in `route_core.py` (pure Python). Edge cost follows §13.2: c = L / max(v, 0.05) · (1 + 2ρ) + w, where v is the agent's nominal speed (robot 0.25, bus/car 0.40 m/s) bounded by the measured mean speed from `/traffic/edges`, ρ the measured density (0 on robot-only links) and w a declared crossing wait (5 s) for edges that intersect C while C is occupied, owned by another agent or unknown. Edges are *removed*, not penalised, when they are forbidden to the agent type (§3.1: parking edges car-only, robot links and detour links robot-only), excluded by the caller, reported blocked or invalid by `traffic_monitor`, within obstacle radius + agent radius + 0.05 m of an obstacle in `/hub/vehicles`, or when their observation is stale (traffic > 1.5 s, obstacles > 1 s) — §13.2 "treat stale as blocked". Dijkstra (ties broken deterministically) returns the path, or `NO_ROUTE`. Each answer is published as a `ROUTE_COMPUTED` event with every edge cost used, and all costs at 1 Hz on `/route/edge_costs`.

Robot (S7). `robot_manager` serves `/robot/escort`, `/robot/navigate_to` and `/robot/confirm_ready` (std_srvs/Trigger); the logic is in `robot_core.py`. States IDLE → TO_PICKUP → WAIT_READY → ESCORT → ARRIVED with the pauses BLOCKED and WAIT_CROSSING and the terminal states CANCELLED, FAILED and SAFE_STOP. Goals are validated (ids, nodes, duration) and deduplicated by request id; a second goal while busy is rejected. At pickup the robot prompts and waits up to 20 s; readiness is accepted only in WAIT_READY. Routes come only from the planner: 1 s timeout, at most 2 retries, then FAILED PLANNER_UNAVAILABLE; answers to an abandoned attempt are ignored (no stale path); a route that cannot meet the mission deadline fails as DEADLINE_INFEASIBLE. While moving, the remaining route is re-validated against the same edge rules every tick: a removed edge ahead stops the robot at the next node and replans from there; if its own edge is removed it returns to the node it came from (reversal along the edge just used). Congestion-only replans happen at most every 2 s and only for ≥ 15 % lower cost. Crossing C is acquired before the footprint reaches it (stop line 0.2 m before C, or immediately when a reversal starts closer), renewed every 1 s and released once clear; the permission is kept across replans while inside C. Arrival = within 0.20 m for 1 s. Odometry older than 0.5 s gives zero command, after 2 s more SAFE_STOP. Progress = travelled / (travelled + remaining on the current leg), so it may decrease after a detour while the remaining distance follows the new route (§14.3).

S6 live (`docker/hub.sh s6`, `evidence/m4/s6/`, 8/8, Figure 7). Car A→E on empty roads: A-Cs-J-E, 25.000 s; the upper detour costs 35.000 s (10 m and 14 m at 0.40 m/s, zero density and wait). Obstacle O removed Cs-J (`OBSTACLE`) and the route became A-N-Cn-B-U-E with no removed edge. Three slow cars on Cs-J raised the lower route to 115 s and the 14 m detour (35 s) was chosen. With O and O2 the car got `NO_ROUTE`, and a robot NavigateTo E failed with `NO_ROUTE` after 0.09 s with no nonzero candidate and 0.000 m movement (T12). After restoring, a new route was computed and validated before the robot moved. T11: all 5 live answers equal an exhaustive simple-path enumeration over the planner's own edge costs, and the unit test repeats this for 20 seeded random edge states (8 queries each) with an independently re-implemented cost function. Planning latency p95 29 ms over 78 calls (limit 250 ms).

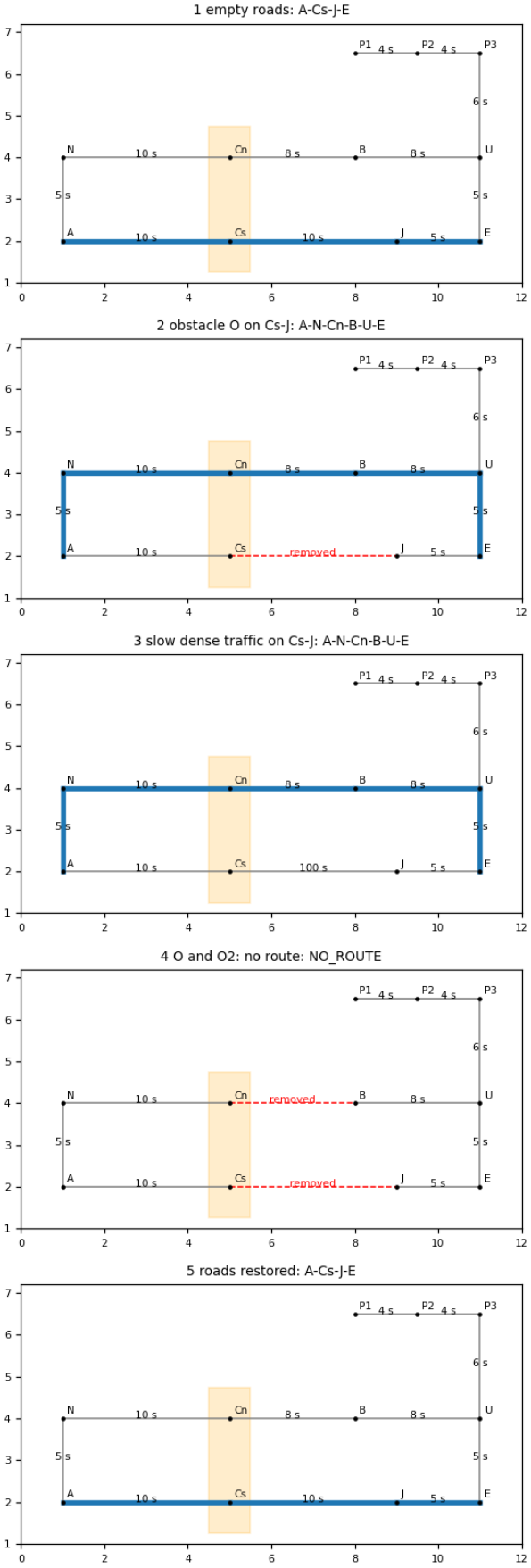


Figure 7 — S6: car edge costs (seconds) and the chosen route (blue) for each step; dashed red = removed edge.

S7 live (`docker/hub.sh s7`, `evidence/m4/s7/`, 9/9, Figure 8). T13: 10/10 escorts R→D via R-W-Cn-Cs-J-D, arrival error ≤ 0.027 m, feedback 5 Hz (largest gap 0.22 s), crossing held whenever the robot was in C, no conflict, no contact. The scenario has no observation noise, so the ten runs are near-identical (escort time 56.8–56.9 s) — they show repeatability, not variety. No readiness: FAILED READY_TIMEOUT after 20.0 s without movement; a readiness press while idle was refused. Obstacle O injected on Cs-J while the robot was on R-W: reroute `W-Cn-B-U-E-J-D`, remaining distance 10.4 → 14.3 m, arrived; escort time 77.9 s versus 56.8 s nominal (paired starts). T14: cancel accepted 0.022 s after the request and zero `/robot/cmd_vel` 0.036 s after acceptance (simulated robot stopped at once — no inertia); a second goal while busy → BUSY; a reused request id → DUPLICATE_REQUEST; destination P1 → NO_ROUTE without movement; odometry frozen → zero candidate after 0.49 s, SAFE_STOP ODOM_STALE after 2.5 s.

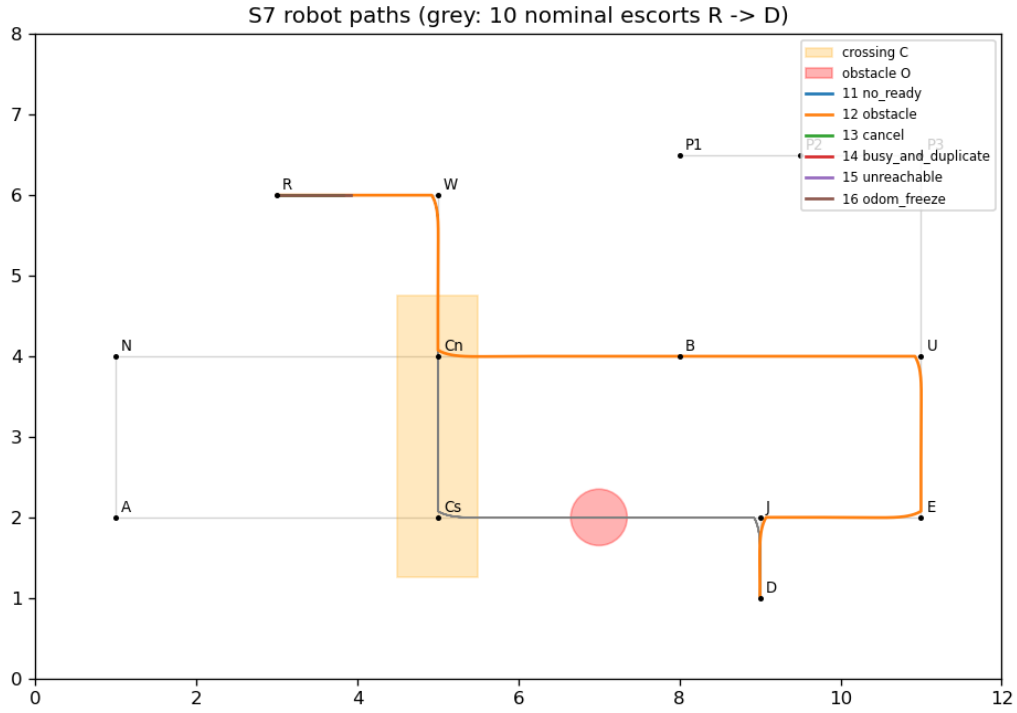


Figure 8 — S7: robot paths. Grey: 10 nominal escorts through C; orange: the reroute around obstacle O (O was active only in that trial).

Unit tests. `test_route_core.py` (28: §13.3 steps 1–4, permissions, stale and unknown edges, crossing wait, segment–rectangle test, T11 × 20 seeds) and `test_robot_core.py` (15: the robot core drives the simulator with no ROS — nominal escort, readiness rules and timeout, obstacle detour, reversal from a blocked current edge back into C with a new grant, cancel, busy, no route, stale odometry, planner absent and late, crossing denied, clearance pause).

Problems found and fixed during development. (1) A NavigateTo to the node the robot already stood on never finished. (2) When a reroute turned the robot back towards C, the normal stop line was already behind it, so it would have entered C without permission; it now holds where it is and asks first. (3) Found by re-running S3 with the planner: in 3 of 15 trials the bus took A-Cs-J-E-U-B and docked facing west (2 nominal dockings failed the yaw check). Cause: the bus parked at A is itself a vehicle at 0 m/s on A-N (or A-Cs, decided by observation noise), so `traffic_monitor` marked its own first edge congested and the planner priced A-N at 80 s. The planner now removes the requesting agent from every edge whose corridor contains it (one vehicle less in the density, its own speed ignored when nobody else is left). A first fix used the vehicle's edge label instead of its position and did not help: at node A the label flips between A-N and A-Cs with observation noise, so the third S3 run still failed the same way and the fix was changed to use the position. (4) The bus fell back to a fixed route when the planner was missing, which violates T18; the bus now uses the same timeout/retry/fail rule (the fallback is an off-by-default parameter). Four test expectations were wrong and were corrected after checking

the behaviour (zero command after the 0.5 s odometry timeout, not instantly; the planning tick has no distance; an obstacle 0.55 m from R-W correctly removes the edge; the reroute measurement window).

Limitations. Congestion replanning is unit-tested but not exercised live. A cancel inside C stops the robot there and keeps the crossing until it leaves. The robot is counted by `traffic_monitor` on shared roads, so standing on Cs-J can briefly mark it congested.

10. M5 — simulator, safety supervisor and infrastructure (S9)

Components. `scenario_runner` (deterministic 20 Hz unicycle simulator, YAML scenarios with seeded events, synthetic camera, fault injection, `/scenario/reset`); `safety_supervisor` (the only publisher of final commands); `actuator_bridge` (independent motor-side watchdog, caps and door interlock); `metrics_logger` (per-run CSV/JSON); `dashboard_marker_publisher` (read-only `/hub/markers`). Each node is a pure-Python core with unit tests plus a thin ROS wrapper.

Safety policy (§16.2, §18.7). Every 50 ms on a monotonic clock the supervisor evaluates: stop latch, e-stop heartbeat, freshness of candidate (0.5 s), odometry and vehicle data (1.0 s), clearance with a 0.5 s motion look-ahead, site boundary, door interlock and crossing permission; it then clamps to 0.25 m/s (robot) and 0.40 m/s (bus). Two kinds of stop:

Kind	Triggers	Ends
Latch (all agents zero)	e-stop true, footprint outside the site, footprint overlap	only `/safety/clear_stop` when every reset rule passes
Hold (affected agent zero)	stale command/odometry/vehicles/e-stop signal, door, clearance, crossing not granted	automatically when the condition is gone

`/safety/clear_stop` is rejected while the e-stop is asserted or stale, a sensor is stale, a hazard remains, measured speed exceeds 0.02 m/s, or a manager still commands motion; clearing never restarts a mission. A `false` e-stop never clears the latch (§7.1).

Three independent stop layers (T17): supervisor (candidate timeout 0.5 s) → actuator bridge (final-command timeout 0.5 s, own caps) → simulator (drive timeout 0.5 s).

Reset handshake (team decision, §7.1 "after all nodes acknowledge reset"). `/scenario/reset` installs the new world, broadcasts `SCENARIO_RESET_REQUEST` on `/hub/events`, re-sends it every 0.25 s, and answers only when every participant sent `RESET_ACK` (else `MISSING_ACK: <names>` after 2 s). The supervisor keeps its latch across a reset.

Results.

Test	Criterion	Measured
T16 stop latency (§21.3)	e-stop → first zero command ≤ 0.20 s	0.4–1.0 ms over 5 trials (measured outside the supervisor); supervisor receipt → zero ≤ 0.14 ms
simulated speed after e-stop	reported separately (§16.3)	below 0.02 m/s after 0.02–0.10 s
S9 recovery (§16.3)	latch, false keeps latch, reset rejected (e-stop / command / stale odometry), accepted when safe, no auto-restart	13/13 checks passed
T17 watchdog	kill controller, then supervisor	controller killed → supervisor zero after 0.52 s; supervisor killed → bridge zero after 0.50 s; bridge killed → simulator stops
fault repetition (§16.3)	each fault 5× at different mission phases	25/25 passed (table below)
silent continuation	no nonzero output while latched	0 occurrences

Fault (5 phases each)	Passed	Zero-command latency mean / max	Simulated stop max	Limit
e-stop	5/5	0.001 / 0.0012 s	0.10 s	0.20 s
candidate stream lost	5/5	0.513 / 0.534 s	0.60 s	0.75 s

Fault (5 phases each)	Passed	Zero-command latency mean / max	Simulated stop max	Limit
odometry frozen	5/5	1.029 / 1.033 s	1.10 s	1.25 s
vehicle data frozen	5/5	1.032 / 1.041 s	1.10 s	1.25 s
e-stop signal lost	5/5	1.022 / 1.039 s	1.10 s	1.25 s

Phases: start (0.3 s after moving off), cruise 0.20 m/s, turn (0.15 m/s, 0.5 rad/s), maximum speed 0.25 m/s, reverse −0.10 m/s; every trial starts from a fresh world through `/scenario/reset`, so the table also shows 25 successful reset handshakes. Latencies for the timeout faults equal their design timeouts, as intended; limits add one 50 ms tick plus transport margin.

Failures found and corrected during development. (1) A logger false positive ("motion while latched") came from cross-topic message ordering; the supervisor now publishes its state before its commands and the metric ignores a 0.1 s ordering window. (2) A multi-threaded executor for the reset handshake slowed the simulator from 20 Hz to about 6 Hz and broke S1/S9; replaced by an asynchronous service in its own callback group. (3) The first asynchronous version deadlocked in the default callback group; the unit test now calls the real service. (4) `ros2 launch` exits 0 even when a check fails; `docker/hub.sh s9` now takes its verdict from the evidence file.

Limits. The synthetic camera draws only people; hardware mode is refused on purpose (§22 requires an independent power cut).

11. Test matrix (T01–T18)

Test	Requirement	Status
T01	S1 counting	PASS (simulation, blob); real video TODO
T02	S1 occlusion / freeze	PASS (simulation)
T03	S2 prediction	PASS (held-out synthetic seeds)
T04	S2 dispatch	pass: live 6/6 (noise, one goal per mission, invalid forecast, feed gap, FALLBACK)
T05, T06	S3 docking, cancel	pass: 10/10 dockings (≤ 0.034 m, ≤ 0.006 rad); cancel → zero 0.001 s
T07–T10	S4, S5	pass: unit tests (M3) and live S4 11/11, S5 9/9 (T07 p95 39 ms, T10 UNKNOWN 0.995 s)
T11–T14	S6, S7	pass: T11 oracle (5 live + 20 seeds), T12 NO_ROUTE with zero motion, T13 10/10 (≤ 0.027 m), T14 busy/no-ready/cancel/stale/unreachable
T15	S8 integration	pass: 20/20 episodes with 0 collisions, 0 crossing conflicts, 0 double allocations, 0 duplicate dispatches; predictive vs reactive on identical seeds
T16	S9 stop latch	PASS (simulation)
T17	S9 watchdog	PASS (simulation, process kill)
T18	S9 service loss	pass: absent and late planner → PLANNER_UNAVAILABLE after 1 s × 3, no motion, late answers ignored (robot and bus)

Automated suite at the evidence commit: `colcon test` 187 tests — 185 passed, 2 expected failures that track M1-B's pending style fixes, 0 errors (`evidence/tests/colcon_test_summary.txt`).

12. Evidence and reproducibility

Everything above can be re-run with Docker from a clean checkout: `docker/hub.sh image`, `build`, `test`, `s9`, `s9x`, `s1`. Seeds are fixed (S9 seed 9, repetition seeds 101–125, S1 seed 1); each run records the commit. Files: `evidence/m5/` (S9 check, metrics, rosbag 13.6 s / 1,784 messages, parameter dumps, 25-trial CSV/JSON, dashboard video and frames), `evidence/m1a/` (S1 checks for both detectors, time series, plot, annotated frames), `src/smart_hub/results/` (M1-B).

13. Limitations

All results are from the simulator; no physical robot, camera or barrier was used, and "stopped" means the simulated velocity. Synthetic people are simple figures, which favours the blob baseline and handicaps HOG. Scenarios S6–S8 are not yet implemented. A bus waiting for a pedestrian in C was demonstrated in S3 (detector → coordinator → bus); the robot equivalent needs S7.

14. Contribution table — TODO (each member fills their own row)

Member	Role	Implemented	Tests owned	Reviewed	How the work was done (declare tools, incl. AI assistance, per course policy)
Husan	M1-A	passenger_detector, perception/, video tools	T01, T02	TODO	TODO
Tuymurod	M1-B	demand_predictor, training pipeline	T03	TODO	TODO
Fazliddin	M2-A	bus_core, bus_manager (written by M5 for M2-A; review by Fazliddin pending)	T05, T06 (S3 checker by M5)	TODO (Fazliddin to confirm)	TODO (Fazliddin to state)
Arkar	M2-B	coordinator_core, hub_coordinator	T04, T15	TODO	TODO
Carla	M3	parking_engine, parking_manager, traffic_engine, traffic_monitor, hub_map; T07–T10 unit tests	T07–T10 (S4/S5 live checkers by M5)	TODO (Carla to confirm)	TODO (Carla to state)
Aidil	M4	route_core, route_planner, robot_core, robot_manager (written by M5 for M4; review by Aidil pending)	T11–T14 (S6/S7 checkers by M5)	TODO (Aidil to confirm)	TODO (Aidil to state)
Saidumar	M5	interfaces, simulator, safety supervisor, actuator bridge, metrics, dashboard, Docker, docs	T16, T17, T18	TODO	TODO